7 Shiny Apps as Packages

The module is based on the principles outlined in Engineering Production-Grade Shiny Apps by Colin Fay (Fay 2020). It serves as a comprehensive guide to building and deploying Shiny applications as packages, following best practices in software engineering.

The organization below is structured as a guide in your learn-to-learn and learn to think alongside the book.

Objectives

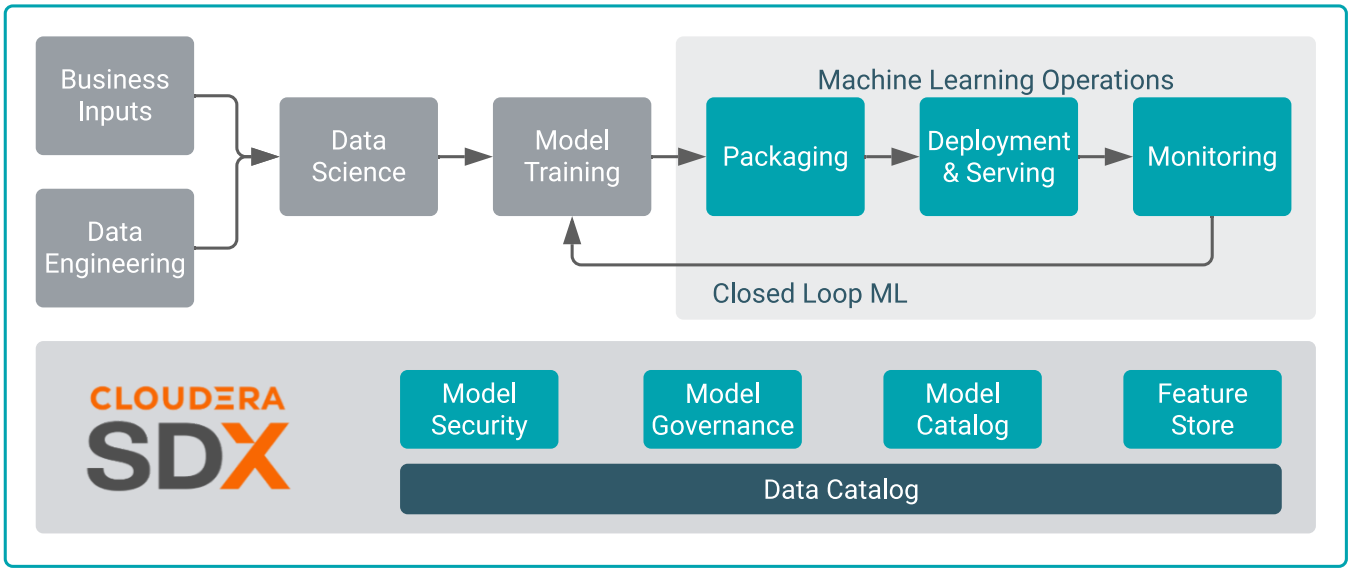
1. Build and deploy shiny apps as packages.
2. Apply the principles of software design to app development.
    - Functional programming
    - Modularization of components
    - Performance optimization
    - Testing

7.1 Why

Shiny applications are a powerful tool for creating interactive web applications in R and python. However, as applications grow in complexity, maintaining and scaling them becomes increasingly challenging.

By building Shiny apps as packages:

- Analysts can leverage the principles of software engineering to create scalable, maintainable, and efficient applications.
- The `{golem}` package provides a framework for building Shiny apps as packages, following best practices in software engineering.
- Enterprises are increasingly using infrastructures like databricks and Cloudera Machine Learning. These provide an integrated environment for data science, data engineering, and production machine learning.
- In this type of infrastructure, every app is deployed as a separate Docker container with full data connectivity. Deployment as a package ensures scalability and maintenance as app are living organisms.



7.2 Section 1: Introduction to `{golem}`

7.2.1 Benefits of using `{golem}`

1. Better structure: `{golem}` enforces a well-defined structure for Shiny apps, making it easier to maintain and scale.
2. Easier deployment: By building Shiny apps as packages, you can deploy them using standard R package deployment tools.
3. Management of namespaces: `{golem}` helps manage namespaces and dependencies, making it easier to integrate with other R packages and tools.

A namespace is a collection of names defined in a package. It is used to prevent name conflicts between packages and to allow for modularization of code.

7.2.2 Setting Up Your Environment

Once you have installed the `{golem}` package using the following command:

```
golem::create_golem(path = "path/to/your/appAsPackage")
```

This will create a new directory with the same structure as a typical R package.

- The `dev` directory contains the development version of the app.
- The `R` directory contains the app's R code, and
- The `inst` directory contains the production version.
- The `inst/app` directory contains the app's UI and server logic.

7.2.3 Testing Your App

From this point onwards, you can use the standard R package development tools to build and test your app. This includes using the `devtools` package to build and test your app, and the `testthat` package to write and run tests.

To debug your app in the built process:

- If you have not yet become comfortable with using the `shiny::browser()` function, you will have to!
- Always ensure it is solely used within a reactive context!
- Don't forget to remove it before deploying your app.

To test your app without building the package:

```
devtools::document()
golem::run_dev()
```

To test your app once the package is built and installed:

```
appName::run_app()
```

7.2.4 Golem Workflow

The `dev` directory contains a set of helper functions to streamline the development process. These include:

1. `01_start.R`: This script sets up the development environment by loading the necessary packages and functions.
2. `02_dev.R`: This script contains the app's UI and server logic, and is used for development and testing.
3. `03_deploy.R`: This script contains the app's UI and server logic, and is used for deployment.
4. `04_test.R`: This script contains the app's test suite, and is used for testing the app's functionality.

7.2.5 Prototyping

The prototyping design pattern involves creating a simple, functional prototype of the app, and then gradually adding features and functionality as needed.

At this stage we are concerned with building the ui logic and basic server logic.

7.2.5.1 UI Prototyping

The core focus is designing the ui without the complexity of the server logic. This is effectively a fully rendered app with placeholder data. From the prior module on Software Design Patters, our goal is to create a simple, functional prototype of the app, and then gradually add features and functionality as needed.

We will use the shinipsum package to generate placeholder text and charts for the app's UI.

Reading material:

- UX Matters
- Building an "ipsum-app"

7.2.5.2 Modularization

To achieve a viable prototype we can build on, we need to modularize our app such that the `app_ui.R` is very clean and rendered via the `mod_<module name>_ui` functions.

See the RTLappDynamics app_ui.R for an example.

- The code below exemplifies the modularization of the app's UI.
- All ui logic for each tab is contained in the module files and the same design patter occurs within each tab.
- Modules are created using the `golem::add_module` function that is part of the set up workflow in the `dev` directory.

```
app_ui <- function(request) {
  shiny::fluidPage(
    golem_add_external_resources(),
    # UI logic
    titlePanel("Commodities Pricing Dynamics"),
    # top of page user input and app intro
    mod_contract_ui("contract_ui_1"),
    # tabset structure
    shiny::tabsetPanel(
     type = "tabs",
     shiny::tabPanel("Forward Curve", mod_CurveDynamics_ui("CurveDynamics_ui_1")),
     shiny::tabPanel("Seasonality", mod_Seasonality_ui("Seasonality_ui_1")),
     shiny::tabPanel("Volatility and Correlation", mod_VolCor_ui("VolCor_ui_1")),
     shiny::tabPanel("Betas", mod_Betas_ui("Betas_ui_1")),
     shiny::tabPanel("Spread Dynamics", mod_SpreadDynamics_ui("SpreadDynamics_ui_1"))
    )
  )
}
```

Reading material: + Using {shiny} modules

7.2.5.3 Learning by Doing

Your challenge, in groups, is to create a `{golem}` app prototype using the `shinipsum` package with multiple tabsets where each tabset has a different dataset and module.

1. Discuss as team what app you want to create: purpose, layout and content.
2. Use the `{golem}` setup workflow to create the module.
3. Create datasets in your package so the skeleton is in place with sensible data. Those datasets are available globally in your package. There is no `global.R` equivalent in the package structure. We will discuss this in the next session.
4. Use the `shinipsum` package to create placeholder text and charts for the app's UI using your package dataset.
5. Create a new repo called `golem_prototype` and push the prototype to the repo.

7.3 Session 2: Advanced `{golem}` Features and the Small r Strategy

7.3.1 Namespaces and Dependencies

When developing Shiny applications, particularly complex or large applications, organizing and managing the UI (user interface) and server logic becomes crucial. This is where the concept of namespaces comes into play, and it's especially important in the context of using the {golem} package in R for building Shiny applications. What are Namespaces?

In the context of Shiny apps, a namespace is a way to uniquely identify UI elements and server logic to prevent name collisions. Name collisions can occur when multiple parts of an app (e.g., modules) use the same identifiers for inputs or outputs. This can lead to unexpected behavior, as Shiny won't be able to distinguish between them correctly.

Namespaces help encapsulate each module, allowing developers to reuse modules without worrying about identifier conflicts. Each module can have its own namespace, making its UI elements and server logic distinct from those of other modules, even if they have the same names. How `{golem}` Utilizes Namespaces

`{golem}` is an R package designed to streamline the process of building robust and maintainable Shiny applications. It encourages a modular approach to app development, where the app is built using separate modules, each responsible for a part of the app's functionality.

- Module Creation: When you create a module, you typically define both a UI function and a server function. `{golem}` encourages the use of namespaces by automatically handling them when you create modules using its conventions.
- UI Namespacing: In the UI part of a module, you use the `ns()` function to apply the module's namespace to UI element IDs. The `ns()` function takes the namespace (usually passed to the UI function as an argument) and an ID, returning a namespaced ID.
- Server Namespacing: On the server side, you use the input and output IDs with the namespace applied. This ensures that the server logic responds to the correct UI elements, even if different modules have elements with the same IDs.
- Using Modules: When you use a module in your Shiny app, you assign it a unique namespace. This is done by providing a namespace string to the module's UI and server functions. Shiny's `NS()` function is often used to generate a namespace function that's passed to the module.

Here's a simplified example of how namespacing works:

```
# Define a UI module function
my_module_ui <- function(id) {
  # Create a namespace function using NS(id). This ensures that all UI element IDs within
  ns <- NS(id)
  tagList(
    # Use the ns function to namespace the ID of the text input. This makes the ID unique
    textInput(ns("text_input"), "Enter something"),
    # Similarly, namespace the ID of the action button.
    actionButton(ns("action_button"), "Click Me")
  )
}

# Define a server module function
```

```
my_module_server <- function(id) {
  moduleServer(id, function(input, output, session) {
    # Observe button clicks. Note: The input IDs are automatically namespaced, so "action
    observeEvent(input$action_button, {
      # Respond to the button click event. For example, print a message using the text in
      message("Button clicked: ", input$text_input)
    })
  })
}

# Define the main app UI
ui <- fluidPage(
  # Incorporate the UI module by calling it with an ID. This ID ("module1") will be used
  my_module_ui("module1")
)

# Define the main server function
server <- function(input, output, session) {
  # Incorporate the server module by calling it with the same ID used in the UI module. T
  my_module_server("module1")
}

# Create the Shiny app with the defined UI and server components
shinyApp(ui, server)
```

In this example, `module1` is the namespace for the module's UI elements. By using `ns()` in the UI and correctly setting up the server logic, the app ensures that inputs and outputs are correctly scoped, even if multiple instances of the module are used with different namespaces.

7.3.2 The `Small r Strategy`

The `Small r Strategy` (SrS) is a design pattern for building Shiny applications using the `{golem}` package. It's a set of best practices and guidelines for structuring Shiny apps to make them more maintainable, scalable, and testable. The SrS encourages a modular approach to app development, where the app is broken down into smaller, reusable components called modules.

- Small r Strategy

A real life and more complex example I developed is RTLappDynamics:

1. The app is modularized into components where each tab is a module.
2. The UI components applying to all modules and the respective computations are in a separate module, mode_contract
3. The SrS design pattern is used by creating a `r` list object this used throughout the app to store the app state and the modules.
4. In order to access the `r` object in the modules, the `r` object is passed to the modules as an argument.

```
mod_CurveDynamics_ui <- function(id){
  ns <- NS(id)
```

```
  tagList(
    shiny::column(12,
                  shiny::plotOutput(ns("fwdCurve"), height = "800px")
                  )
  )
}

# the r object is passed to the module along with the id
mod_CurveDynamics_server <- function(id, r) {

  moduleServer(id,
               function(input, output, session) {
                 output$fwdCurve <-  renderPlot({
                   # acessing the r object within a reactive context.
                   df <- r$datWide
                   cmdty <- r$cmdty
                   RTL::chart_fwd_curves(
                     df = df,
                     cmdty = cmdty,
                     weekly = TRUE,
                     main = "Forward Curves",
                     ylab = ifelse(r$contract == "NG","$ per mmBTU","$ per bbl"),
                     xlab = "",
                     cex = 2
                   )
                 })
               })
}
```

You also need to modify your `app_server.R` file to pass the `r` object:

```
app_server <- function(input, output, session) {
  r <- shiny::reactiveValues()
  mod_compute_server("compute_1", r = r)
  }
```

7.3.3 Performance and Debottlenecking

When developing Shiny apps, it's important to consider performance optimization and debottlenecking strategies to ensure that the app runs smoothly and efficiently. Here are some tips and techniques to improve the performance of your Shiny apps:

- Using the `profvis` package to profile the app and identify bottlenecks in the code.
- Avoiding unnecessary reactivity by using `isolate()` and `observeEvent()` to prevent reactive expressions from being re-evaluated unnecessarily.
- Minimizing data transfer between the server and the client by using `reactive()` and `reactiveValues()` to store intermediate results and avoid repeated computations.

You can run following command and interact with your app. `profvis` will record the performance of your app and generate a visual profile that highlights the bottlenecks in the code.

```
# when running on a golem app, you need to wrap the argument in a print() function
profvis::profvis({print(RTLappDynamics::run_app())})
```

7.3.4 Enhancing Your App with Advanced Features

Once you have mastered the basics of shiny app as packages development, it's time to augment your skills on the UI side!

I strongly encourage you to explore Granjon (2022) for a comprehensive guide on how to create outstanding UIs in Shiny apps. The book covers a wide range of topics, including customizing UI elements with HTML and CSS and using using JavaScript to enhance interactivity. We will cover a brief introduction on the latter during the last class.

7.3.5 Learning by Doing

Building on the previous app you built last week:

1. Think of the modules and user interface components that can be modularized.
2. Create a module with the SrS design pattern, watching for `Observers` events when you object is a function of multiple inputs.
3. Use the `r` object across the modules.
4. Run profvis to identify bottlenecks in your app.